

DENNIS GUNAWAN



UMN
UNIVERSITAS
MULTIMEDIA
NUSANTARA

IF351 DATABASE SYSTEMS

04 THE RELATIONAL DATA MODEL, RELATIONAL DATABASE CONSTRAINTS, AND RELATIONAL DATABASE DESIGN



REVIEW:

DATA MODELING USING THE ENTITY-RELATIONSHIP MODEL

- Database Design
- Entities
- Attributes
- Relationships
- ER Diagrams Notation

COURSE SUB LEARNING OUTCOMES (SUB-CLO)

- Sub-CLO 3
 - Students are able to explain relational data model and relational database constraints (C2)
- Sub-CLO 4
 - Students are able to apply ER-to-Relational Mapping Algorithm to design a relational database (C3)

OUTLINE

- Relational Data Model
- Relational Database Constraints
- ER-to-Relational Mapping Algorithm

RELATIONAL MODEL CONCEPTS

- The relational model represents the database as a collection of relations
- Each relation resembles a table of values
- Each row in the table represents a collection of related data values
- Each row in the table represents a fact that typically corresponds to a real-world entity or relationship
- The table name and column names are used to help to interpret the meaning of the values in each row

RELATIONAL MODEL CONCEPTS

- A row is called a **tuple**
- A column header is called an **attribute**
- The table is called a **relation**

The diagram illustrates the components of a relation. The relation name 'STUDENT' is shown above the table. Arrows point from the label 'Relation Name' to 'STUDENT' and from 'Attributes' to each of the seven column headers. On the left, an arrow labeled 'Tuples' points to the first five rows of the table.

STUDENT						
Name	Ssn	Home_phone	Address	Office_phone	Age	Gpa
Benjamin Bayer	305-61-2435	(817)373-1616	2918 Bluebonnet Lane	NULL	19	3.21
Chung-cha Kim	381-62-1245	(817)375-4409	125 Kirby Road	NULL	18	2.89
Dick Davidson	422-11-2320	NULL	3452 Elgin Road	(817)749-1253	25	3.53
Rohan Panchal	489-22-1100	(817)376-9821	265 Lark Lane	(817)749-6492	28	3.93
Barbara Benson	533-69-1238	(817)839-8461	7384 Fontana Lane	NULL	19	3.25

DOMAINS

- The data type describing the types of values that can appear in each column is represented by a domain of possible values

- “Gender” = {Male, Female}
- “GradePointAverages” must be a real (floating-point) number between 0 and 4

RELATION SCHEMA

- A relation schema R , denoted by $R(A_1, A_2, \dots, A_n)$, is made up of a relation name R and a list of attributes $A_1, A_2, \dots, A_n$

- STUDENT(Name, Ssn, Home_phone, Address, Office_phone, Age, Gpa)
- STUDENT(Name: string, Ssn: string, Home_phone: string, Address: string, Office_phone: string, Age: integer, Gpa: real)

RELATIONAL DATABASE SCHEMA

EMPLOYEE

Fname	Minit	Lname	<u>Ssn</u>	Bdate	Address	Sex	Salary	Super_ssn	Dno
-------	-------	-------	------------	-------	---------	-----	--------	-----------	-----

DEPARTMENT

Dname	<u>Dnumber</u>	Mgr_ssn	Mgr_start_date
-------	----------------	---------	----------------

DEPT_LOCATIONS

<u>Dnumber</u>	<u>Dlocation</u>
----------------	------------------

PROJECT

Pname	<u>Pnumber</u>	Plocation	Dnum
-------	----------------	-----------	------

WORKS_ON

<u>Essn</u>	<u>Pno</u>	Hours
-------------	------------	-------

DEPENDENT

<u>Essn</u>	<u>Dependent_name</u>	Sex	Bdate	Relationship
-------------	-----------------------	-----	-------	--------------

RELATIONAL MODEL CONSTRAINTS

Domain Constraints

- Domain constraints specify that within each tuple, the value of each attribute A must be an atomic value from the domain $\text{dom}(A)$

Key Constraints

- All tuples in a relation must be distinct
- No two tuples can have the same combination of values for all their attributes
- A relation schema may have more than one key, each of the keys is called a **candidate key**
- It is common to designate one of the candidate keys as the **primary key** of the relation
- Primary key is the candidate key whose values are used to identify tuples in the relation

RELATIONAL MODEL CONSTRAINTS

CAR

<u>License_number</u>	Engine_serial_number	Make	Model	Year
Texas ABC-739	A69352	Ford	Mustang	02
Florida TVP-347	B43696	Oldsmobile	Cutlass	05
New York MPO-22	X83554	Oldsmobile	Delta	01
California 432-TFY	C43742	Mercedes	190-D	99
California RSK-629	Y82935	Toyota	Camry	04
Texas RSK-629	U028365	Jaguar	XJS	04

RELATIONAL MODEL CONSTRAINTS

Constraint on NULL Values

- Specifies whether NULL values are or are not permitted
- For example, if every STUDENT tuple must have a valid, non-NULL value for the Name attribute, then Name of STUDENT is constrained to be NOT NULL

Entity Integrity Constraint

- No primary key value can be NULL
- This is because the primary key value is used to identify individual tuples in a relation
- Having NULL values for the primary key implies that we cannot identify some tuples

RELATIONAL MODEL CONSTRAINTS

Referential Integrity Constraint

- A tuple in one relation that refers to another relation must refer to an existing tuple in that relation
- **Foreign key** is an attribute or a set of attributes of a relation whose values are required to match the values of the primary key of another relation
- A set of attributes FK in relation schema R_1 is a foreign key of R_1 that references relation R_2 if it satisfies the following rules
 - The attributes in FK have the **same domain(s)** as the primary key attributes PK of R_2
 - A value of FK in a tuple t_1 of the current state $r_1(R_1)$ either **occurs as a value of PK** for some tuple t_2 in the current state $r_2(R_2)$ or is **NULL**

RELATIONAL MODEL CONSTRAINTS

EMPLOYEE

Fname	Minit	Lname	<u>Ssn</u>	Bdate	Address	Sex	Salary	Super_ssn	Dno
-------	-------	-------	------------	-------	---------	-----	--------	-----------	-----

DEPARTMENT

Dname	<u>Dnumber</u>	Mgr_ssn	Mgr_start_date
-------	----------------	---------	----------------

DEPT_LOCATIONS

<u>Dnumber</u>	<u>Dlocation</u>
----------------	------------------

PROJECT

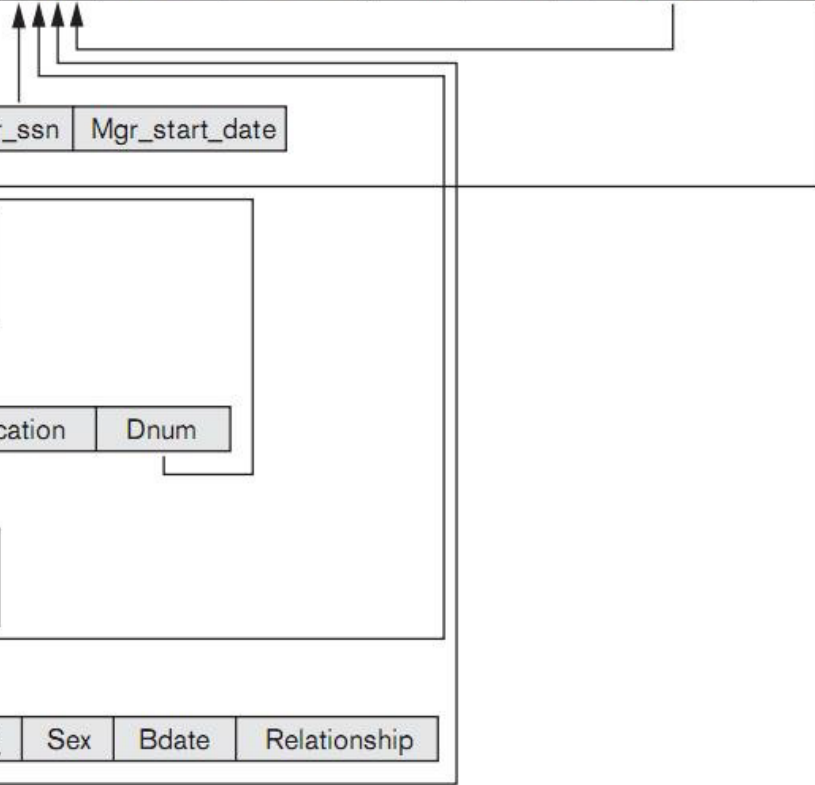
Pname	<u>Pnumber</u>	Plocation	Dnum
-------	----------------	-----------	------

WORKS_ON

<u>Essn</u>	<u>Pno</u>	Hours
-------------	------------	-------

DEPENDENT

<u>Essn</u>	<u>Dependent_name</u>	Sex	Bdate	Relationship
-------------	-----------------------	-----	-------	--------------



MODIFICATION OPERATIONS & DEALING WITH CONSTRAINT VIOLATIONS

- Three basic modification operations on relations
 - **Insert** is used to insert a new tuple or tuples in a relation
 - **Delete** is used to delete tuples
 - **Update** is used to change the values of some attributes in existing tuples
- Whenever these operations are applied, the integrity constraints specified on the relational database schema should not be violated

MODIFICATION OPERATIONS & DEALING WITH CONSTRAINT VIOLATIONS

Insert Operation

- **Domain constraints** can be violated if an attribute value is given that does not appear in the corresponding domain
- **Key constraints** can be violated if a key value in the new tuple already exists in another tuple in the relation
- **Entity integrity** can be violated if the primary key of the new tuple is NULL
- **Referential integrity** can be violated if the value of any foreign key in new tuple refers to a tuple that does not exist in the referenced relation
- If an insertion violates one or more constraints, the default option is to **reject the insertion** (DBMS could explain to the user why the insertion was rejected)

MODIFICATION OPERATIONS & DEALING WITH CONSTRAINT VIOLATIONS

Delete Operation

- Delete operation can violate only **referential integrity** if the tuple being deleted is referenced by the foreign keys from other tuples in the database
- Several options are available if a deletion operation causes a violation
 - Reject the deletion
 - Attempt to cascade the deletion by deleting tuples that reference the tuple that is being deleted
 - Modify the referencing attribute values that cause the violation (set to NULL or changed to reference another valid tuple)

MODIFICATION OPERATIONS & DEALING WITH CONSTRAINT VIOLATIONS

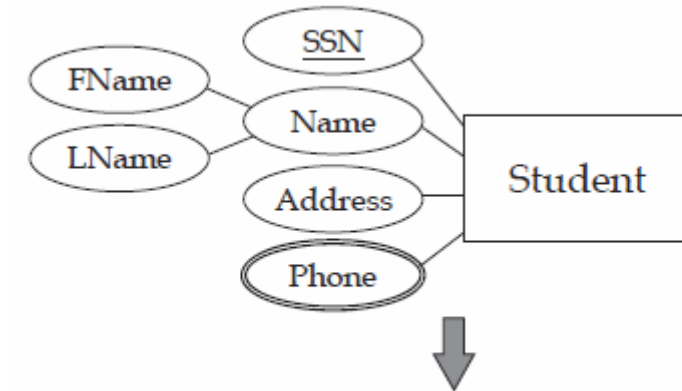
Update Operation

- Updating an attribute that is neither a primary key nor a foreign key can violate only the domain constraint
- Updating a primary key value is similar to deleting one tuple and inserting another in its place
- If a foreign key attribute is updated, the DBMS must make sure that the new value refers to an existing tuple in the referenced relation

ER-TO-RELATIONAL MAPPING ALGORITHM

Step 1: Mapping of Regular Entity Types

- For each regular (strong) entity type E in the ER schema, create a relation R that includes all the simple attributes of E
- Include only the simple component attributes of a composite attribute
- Choose one of the key attributes of E as the primary key for R (the primary key of E is the primary key of R)

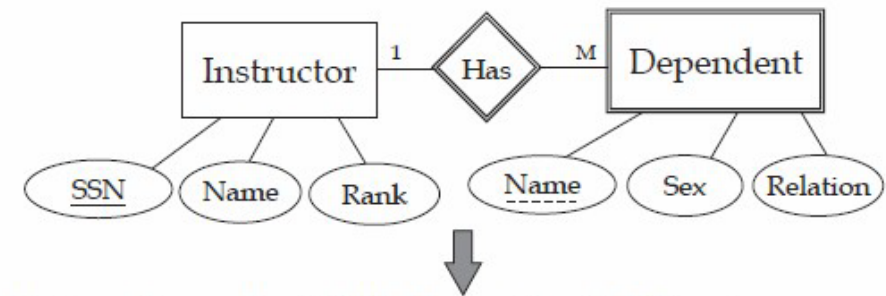


Relation: Student(SSN, FName, LName, Address)

ER-TO-RELATIONAL MAPPING ALGORITHM

Step 2: Mapping of Weak Entity Types

- For each weak entity type W in the ER schema with owner entity type E , create a relation R and include all simple attributes (or simple components of composite attributes) of W as attributes of R
- Include the primary key attribute(s) of E as foreign key attribute(s) of R
- The primary key of R is the combination of the primary key of E and the partial key of W , if any

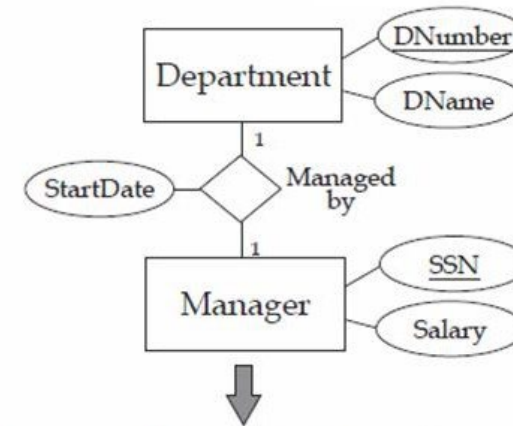


Relation: `Dependent(SSN, Name, Sex, Relation)`
where SSN in Dependent is a foreign key which references the Instructor Relation.

ER-TO-RELATIONAL MAPPING ALGORITHM

Step 3: Mapping of Binary 1:1 Relationship Types

- For each binary 1:1 relationship type R in the ER schema, identify the relations S and T that correspond to the entity types participating in R
- Choose one of the relations (for example, S) and include the primary key of T as a foreign key in S
- Include all the simple attributes (or simple components of composite attributes) of the 1:1 relationship type R as attributes of S



Relations: Department(DNumber, DName)
Manager(SSN, Salary, DNumberManaged, StartDate)
where DNumberManaged in Manager is a foreign key which
references the Department relation.

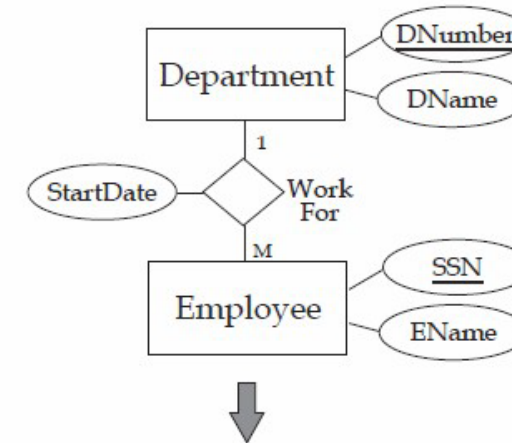
or

Department(DNumber, DName, MgrSSN, StartDate)
Manager(SSN, Salary)
where MgrSSN in Department is a foreign key which references
the Manager relation.

ER-TO-RELATIONAL MAPPING ALGORITHM

Step 4: Mapping of Binary 1:N Relationship Types

- For each regular binary 1:N relationship type R , identify the relation S that represents the participating entity type at the N-side of the relationship type
- Include the primary key of the relation T as foreign key in S
- Include any simple attributes (or simple components of composite attributes) of the 1:N relationship type as attributes of S

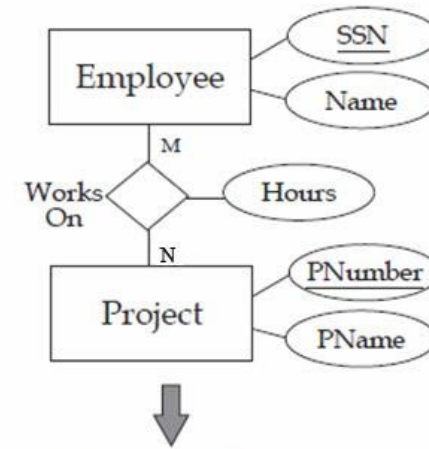


Relations: Department(DNumber, DName)
Employee(SSN, EName, DNumber, StartDate)
where DNumber in Employee is a foreign key which references
the Department relation

ER-TO-RELATIONAL MAPPING ALGORITHM

Step 5: Mapping of Binary M:N Relationship Types

- For each binary M:N relationship type R , create a new relation S to represent R
- Include the primary keys of the relations that represent the participating entity types as foreign key attributes in S
- Include any simple attributes of the M:N relationship type (or simple components of composite attributes) as attributes of S

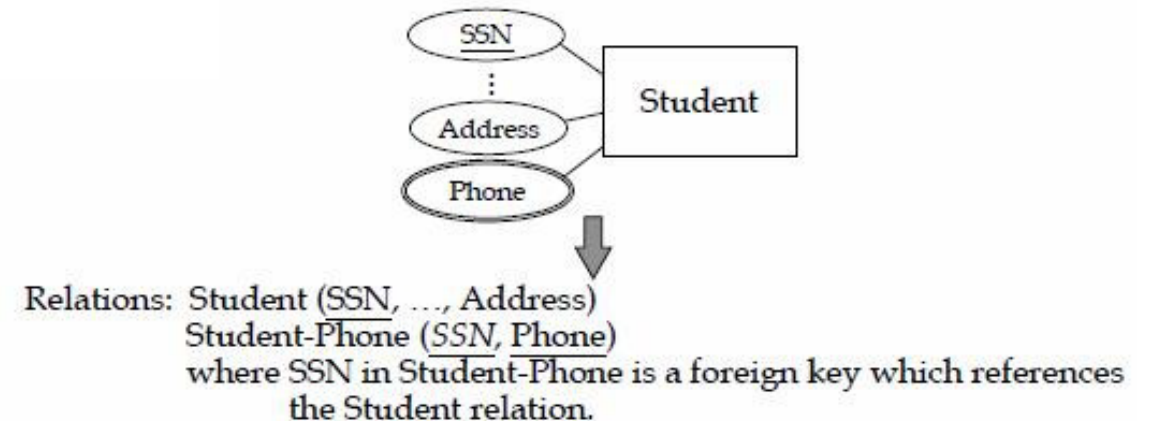


Relations: Employee(SSN, EName, DNumber, StartDate)
Project(PNumber, PName)
Works-On(SSN, PNumber, Hours)
where SSN and PNumber in Works-On are foreign keys which reference, respectively, the Employee relation and the Project relation.

ER-TO-RELATIONAL MAPPING ALGORITHM

Step 6: Mapping of Multivalued Attributes

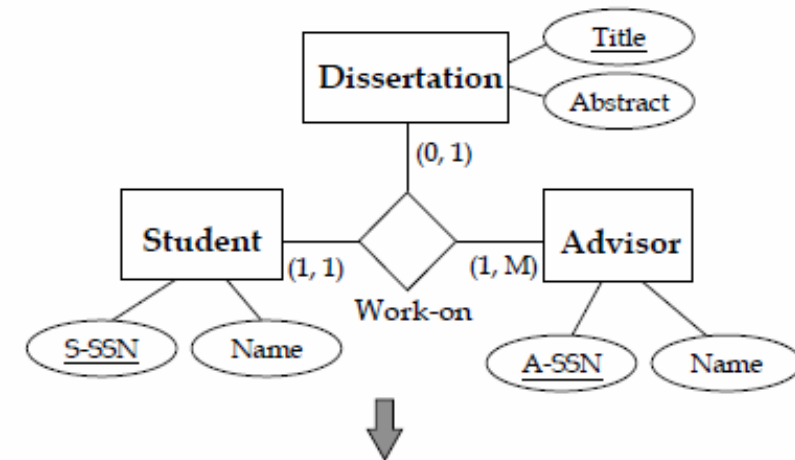
- For each multivalued attribute A , create a new relation R
- Relation R will include an attribute corresponding to A , plus the primary key attribute K (as a foreign key in R) of the relation that represents the entity type or relationship type that has A as an attribute
- The primary key of R is the combination of A and K



ER-TO-RELATIONAL MAPPING ALGORITHM

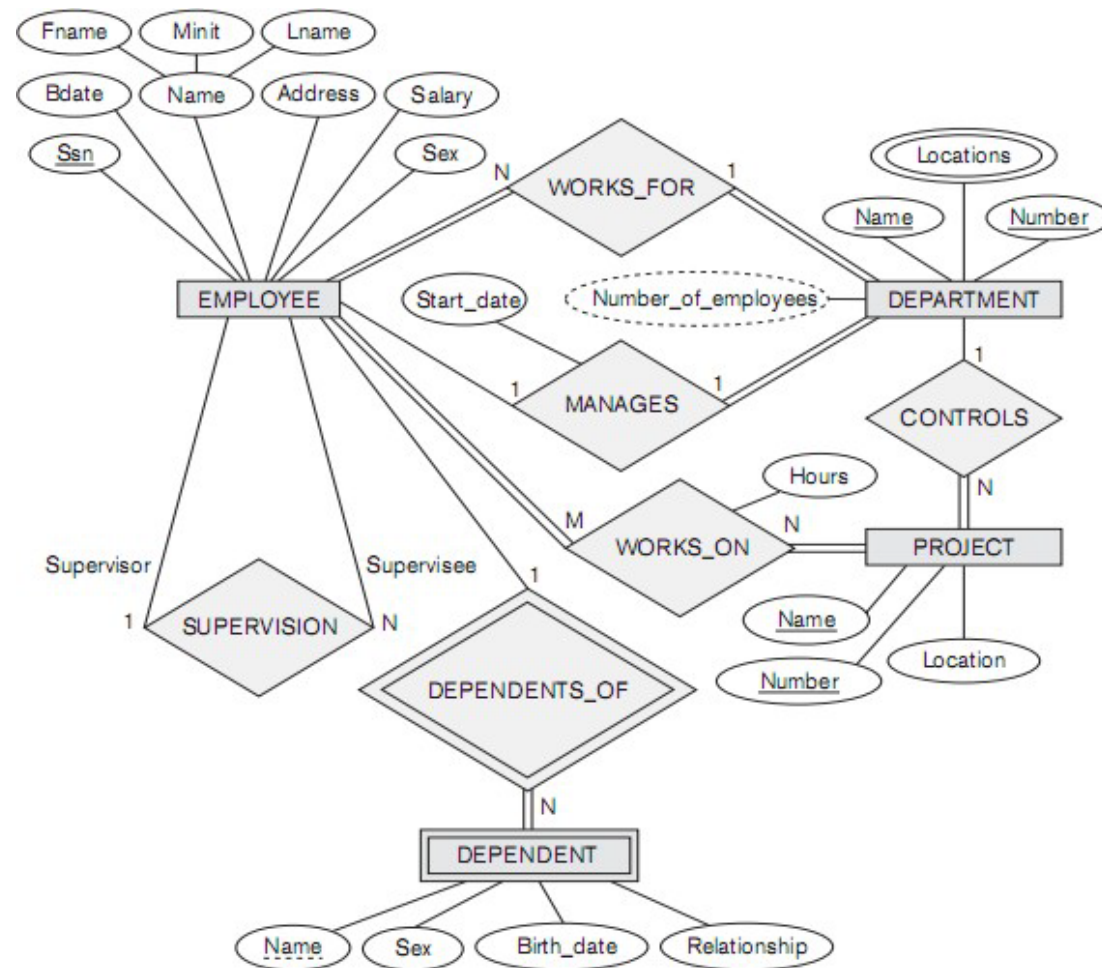
Step 7: Mapping of N -ary Relationship Types

- For each n -ary relationship type R , where $n > 2$, create a new relation S to represent R
- Include the primary keys of the relations that represent the participating entity types as foreign key attributes in S
- Include any simple attributes of the n -ary relationship type (or simple components of composite attributes) as attributes of S
- The primary key of S is usually a combination of all the foreign keys that reference the relations representing the participating entity types



Relations: Student(S-SSN, Name)
Advisor(A-SSN, Name)
Dissertation(Title, Abstract)
Work-on(S-SSN, A-SSN, Title)
where S-SSN, A-SSN and Title in Work-on are foreign keys
reference, respectively, the Student, Advisor, and
Dissertation relations.

EXAMPLE



EXAMPLE

EMPLOYEE

Fname	Minit	Lname	<u>Ssn</u>	Bdate	Address	Sex	Salary	Super_ssn	Dno
-------	-------	-------	------------	-------	---------	-----	--------	-----------	-----

DEPARTMENT

Dname	<u>Dnumber</u>	Mgr_ssn	Mgr_start_date
-------	----------------	---------	----------------

DEPT_LOCATIONS

<u>Dnumber</u>	<u>Dlocation</u>
----------------	------------------

PROJECT

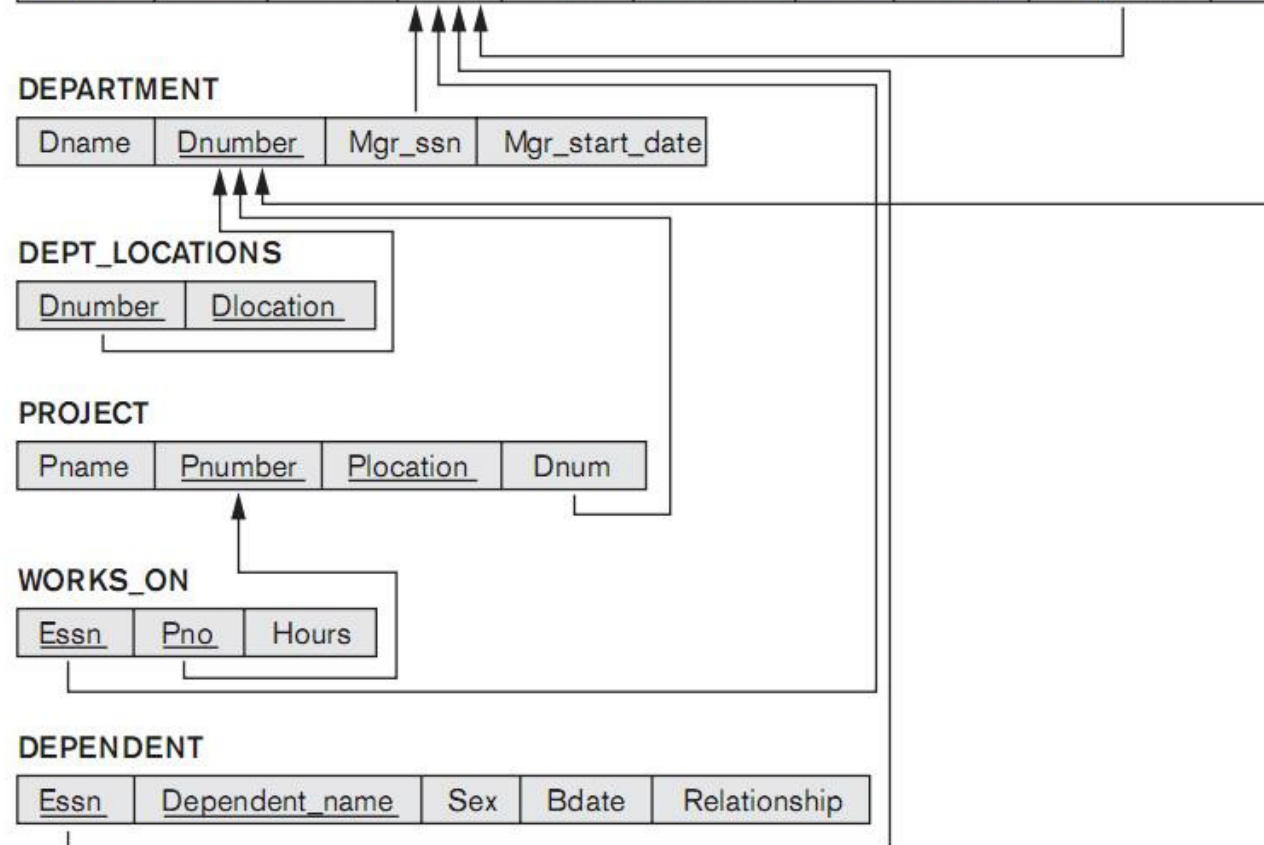
Pname	<u>Pnumber</u>	<u>Plocation</u>	Dnum
-------	----------------	------------------	------

WORKS_ON

<u>Essn</u>	<u>Pno</u>	Hours
-------------	------------	-------

DEPENDENT

<u>Essn</u>	<u>Dependent_name</u>	Sex	Bdate	Relationship
-------------	-----------------------	-----	-------	--------------



REFERENCES

- Elmasri, Ramez and Shamkant B. Navathe (2016), Fundamentals of Database Systems, 7th edition, Addison Wesley.
- Rick F.van der Lans (2007), SQL for MySQL Developers, Addison Wesley.
- Connolly, Thomas and Carolyn Begg (2015), Database Systems A Practical Approach to Design, Implementation, and Management, 6th edition, Pearson Education, ISBN 13: 978-1-292-06118-4, Essex, England.

NEXT WEEK:

DATA DEFINITION LANGUAGE (DDL) AND DATA MANIPULATION LANGUAGE (DML) STATEMENTS

- Creating, Showing, & Deleting D atabases
- Selecting the Current D atabase
- Data Types of Columns
- Creating, Showing, & Deleting Tables
- Populating Tables with Data
- Querying Tables
- Updating & Deleting Rows
- Copying Tables
- Populating a Table with Rows from Another Table

```
SELECT *  
FROM WORLD  
WHERE SOMEONE  
LIKE '%YOU%';
```

```
EMPTY SET (0.00 SEC)
```

Vision

To become an **outstanding** undergraduate Computer Science program that produces **international-minded** graduates who are **competent** in software engineering and have **entrepreneurial spirit** and **noble character**.



Mission

1. To conduct studies with the best technology and curriculum, supported by professional lecturer
2. To conduct research in Informatics to promote science and technology
3. To deliver science-and-technology-based society services to implement science and technology